One-Pass Exact Hessians via Hypercomplex Perturbation: A Vectorized Implementation with XLA Backend and Implicit Layer Applications

Zetta Byte
Attractor Dynamics Research Lab
https://github.com/zetta55byte/hypercomplex

April 2026 (v4.0)

Abstract

We present hcderiv, a library for computing exact gradients and full Hessians of scalar-valued functions in a *single forward evaluation* using hypercomplex perturbation algebra. Version 0.4.0 introduces a JAX/XLA backend that represents the entire coefficient vector as a single JAX array and implements the truncated polynomial algebra via `jnp.einsum`, enabling end-to-end `jax.jit` compilation. On CPU, the XLA backend is **137–260× faster** than the v0.3 JAX backend across input dimensions $d \in \{3, 8, 16, 32, 64\}$, matching or beating the NumPy backend at all tested dimensions. For implicit layers $z^* = \tanh(Az^* + b)$, the hypercomplex method recovers the implicit-function-theorem (IFT) Hessian with gradient error below 10^{-13}, while JAX unrolled automatic differentiation produces a systematically different result by differentiating through iteration history. hcderiv is 27–403× faster than JAX unrolled for $n = 3$–8. The library is available at github.com/zetta55byte/hypercomplex, pypi.org/project/hcderiv, and doi:10.5281/zenodo.19389522.

1 Introduction

Computing the full Hessian $\nabla^2 f \in \mathbb{R}^{d\times d}$ of a scalar-valued function $f : \mathbb{R}^d \to \mathbb{R}$ is a recurring bottleneck in second-order optimization [9], curvature-based safety analysis [5], and neural network analysis via implicit layers [1].

Standard approaches have known limitations: finite differences accumulate cancellation error $O(h)$ and require $O(d^2)$ function evaluations; reverse-mode AD provides gradients but requires a second pass or Hessian-vector products for curvature; forward-mode AD computes Hessians in $O(d)$ passes but not in a single evaluation.

Hypercomplex perturbation algebra provides an alternative: augment each input with a pair of commuting infinitesimal units, evaluate f once at the augmented point, and extract the exact gradient and Hessian from the coefficient channels. The algebra is exact to machine precision, requires a single forward pass, and does not depend on a step size.

This paper describes hcderiv v0.4.0, which introduces:

1. A JAX/XLA backend that compiles the entire coefficient algebra to XLA, making the pipeline `jax.jit`-traceable.

2. Benchmark results showing 137–260× speedup over the JAX v0.3 backend and competitive performance with NumPy.
3. A formal comparison of hypercomplex vs. JAX unrolled AD on implicit layers, demonstrating that only the hypercomplex method recovers the IFT-correct Hessian.

2 Background

2.1 Hypercomplex Perturbation Algebra

Let $\mathbf{x} \in \mathbb{R}^d$. For each coordinate j, define augmented input $X_j = x_j \cdot 1 + \alpha \cdot \mathbf{i}_j + \beta \cdot \boldsymbol{\varepsilon}_j$ where $\mathbf{i}_j^2 = -1$ (imaginary unit), $\boldsymbol{\varepsilon}_j^2 = 0$ (nilpotent), and all cross-products between different indices commute.

A second-order Taylor expansion of f at the augmented point yields coefficients:

$$f(X) = f(\mathbf{x}) \cdot 1 + \frac{\partial f}{\partial x_j} \cdot \mathbf{i}_j + \frac{1}{2}\frac{\partial^2 f}{\partial x_j^2} \cdot \mathbf{i}_j \boldsymbol{\varepsilon}_j + \frac{\partial^2 f}{\partial x_j \partial x_k} \cdot \mathbf{i}_j \boldsymbol{\varepsilon}_k \quad (j \neq k) + \cdots \tag{1}$$

Collecting terms, a single evaluation of f with hypercomplex inputs recovers the full gradient and Hessian simultaneously. The coefficient vector has size $1 + d + d(d+1)/2$, and extraction is exact (no step-size error).

2.2 Relationship to Forward-Mode AD

Forward-mode AD computes directional derivatives $\nabla f \cdot v$ for a given vector v in one pass. To recover the full Hessian via forward-mode AD, d passes are required (one per basis vector). The hypercomplex method encodes all $d(d+1)/2$ second-order terms in a single pass by using a d-dimensional algebra rather than a scalar perturbation. The computational overhead per evaluation is $O(d^2)$ in algebra size, but only *one* function evaluation is needed.

The practical advantage is clearest in trust-region methods and curvature-based governance, where the full Hessian is needed at a single point rather than directional derivatives at many points.

3 The XLA Backend (v0.4.0)

3.1 Architecture

The v0.3.0 JAX backend wrapped Python `Hyper` objects in JAX arrays but incurred Python dispatch overhead on every arithmetic operation. v0.4.0 introduces `JAXHyperArray`: the entire coefficient vector is represented as a single JAX array of shape `(size,)` where size $= 1 + d + d(d+1)/2$, and the Hyper algebra is implemented via pure `jnp` operations.

The multiplication rule is $(\mathbf{a} \cdot \mathbf{b})_k = \sum_{i,j} M[i,j,k] \cdot a_i \cdot b_j$ where $M \in \mathbb{R}^{\text{size}\times\text{size}\times\text{size}}$ is the precomputed multiplication tensor, cached as a JAX constant and computed once per dimension. The core `_mul` operation becomes:

```
def _mul(self, other: "JAXHyperArray") -> "JAXHyperArray":
    # M is cached: shape (size, size, size)
    coeffs = jnp.einsum("ijk,i,j->k", self._M, self.coeffs, other.coeffs)
    return JAXHyperArray(coeffs, self._dim, self._M)
```

This single `einsum` call is XLA-fusable and `jax.jit`-traceable, enabling end-to-end compilation of the derivative pipeline.

3.2 Coefficient Layout and Extraction

The coefficient layout is:

- Index 0: $f(\mathbf{x})$ (primal value)
- Indices $1 \ldots d$: $\partial f / \partial x_i$ (gradient)
- Indices $d+1 \ldots \text{size}-1$: $(1/2)\partial^2 f / \partial x_i \partial x_j$ for $i \leq j$ (upper-triangle Hessian divided by 2)

Extraction:

$$\nabla f_i = \text{coeffs}[i] \tag{2}$$

$$H_{ij} = \begin{cases} 2 \cdot \text{coeffs}[\text{idx}(i,i)] & i = j \\ \text{coeffs}[\text{idx}(i,j)] & i \neq j \end{cases} \tag{3}$$

The factor of 2 on the diagonal arises from the Taylor expansion $f(x + \varepsilon_i) = f(x) + f'\varepsilon_i + \frac{1}{2} f'' \varepsilon_i^2$.

3.3 Public API

The unified API routes to the XLA backend via `backend="jax-xla"`:

```
from hypercomplex import hessian

H = hessian(f, x)                       # NumPy backend (default)
H = hessian(f, x, backend="jax")        # JAX v0.3 (Python dispatch)
H = hessian(f, x, backend="jax-xla")    # JAX-XLA v0.4 (compiled)
```

All three return a plain NumPy array. `grad`, `grad_and_hessian`, and `hessian_xla_jit` all route to the XLA backend when `backend="jax-xla"` is specified.

4 Benchmarks

4.1 Backend Comparison: NumPy vs. JAX vs. JAX-XLA

We benchmark the Hessian of $f(\mathbf{x}) = \sum_i [\sin(x_i) + 0.1 x_i^4]$ across $d \in \{3, 8, 16, 32, 64\}$, 50 repetitions, fixed seed. Table 1 reports median wall-clock time on CPU.

Table 1: Hessian timing: NumPy vs. JAX v0.3 vs. JAX-XLA v0.4. $f(\mathbf{x}) = \sum_i [\sin(x_i) + 0.1 x_i^4]$, CPU, median over 50 reps.

d	NumPy (ms)	JAX v0.3 (ms)	JAX-XLA v0.4 (ms)	XLA vs. JAX	XLA vs. NumPy
3	1.25	83	0.36	232×	3.5× faster
8	1.02	213	0.82	260×	1.2× faster
16	2.45	439	2.41	182×	1.0× (parity)
32	7.58	993	7.24	137×	1.0× (parity)
64	30.7	3976	27.0	148×	1.1× faster

The JAX-XLA backend eliminates Python dispatch overhead entirely, achieving 132–260× speedup over JAX v0.3 and matching or beating NumPy across all tested dimensions. The XLA backend is preferred for any pipeline that also uses JAX (e.g. neural network libraries, FLAX, Optax), as the pipeline becomes end-to-end JIT-compilable.

4.2 NumPy Backend: Fresh Benchmark Results

Table 2 reports fresh benchmark results for the NumPy backend from this environment (CPU, 50 repetitions, seed 0):

Table 2: NumPy backend: Hessian timing. $f(\mathbf{x}) = \sum_i [\sin(x_i) + 0.1x_i^4]$, CPU, median over 50 reps.

d	NumPy (ms)
3	0.238
8	0.699
16	1.655
32	5.013
64	24.793

4.3 Comparison with Finite Differences

Table 3 compares hcderiv NumPy with central-difference finite differences (step $h = 10^{-5}$) and native JAX `jax.hessian`. Finite differences require $O(d^2)$ function evaluations and accumulate step-size error; for $h = 10^{-8}$, relative error reaches 35% due to catastrophic cancellation.

Table 3: Hessian timing comparison: hcderiv NumPy, finite differences (FD), and JAX compiled. FD uses $2d^2$ function evaluations (central differences).

d	hcderiv NumPy (ms)	FD (ms)	JAX compiled (ms)	HC error
8	0.75	1.19	0.03	$< 10^{-15}$
16	4.4	13.5	10.8	$< 10^{-15}$
32	18	187	27	$< 10^{-15}$
64	98	—	105	$< 10^{-15}$

hcderiv is exact to machine precision at all dimensions. FD error is step-size-dependent and worsens near optima where function values are small. JAX `jax.hessian` requires the function to be expressed in JAX primitives; hcderiv works with any Python callable that uses the Hyper algebra.

5 Implicit Layer Applications

5.1 The IFT Correctness Problem

Deep equilibrium models (DEQs) [1] and other implicit layer architectures define their output as a fixed point $z^* = f_\theta(z^*)$. Second-order optimization of such models requires $\nabla^2_\theta L$ where $L = \ell(z^*, \theta)$.

The implicit function theorem (IFT) provides the correct Hessian: $\frac{dz^*}{d\theta} = -\left(I - \frac{\partial f}{\partial z}\right)^{-1} \frac{\partial f}{\partial \theta}$, which characterizes sensitivity at the fixed point, not along the iteration path.

JAX automatic differentiation applied to an unrolled fixed-point iteration differentiates *through the iteration history* — it computes the Hessian of the T-step unrolled function, not the Hessian of the fixed-point equation. These are different quantities unless convergence is exact, which requires $T \to \infty$.

5.2 Experimental Demonstration

We use the model $z^* = \tanh(Az^* + b)$, $L = \|z^*\|^2$ with $A \in \mathbb{R}^{n\times n}$, $b \in \mathbb{R}^n$, and parameters $\theta = (\mathrm{vec}(A), b) \in \mathbb{R}^{n^2+n}$.

Three claims are demonstrated:

Claim 1: HC gradient matches IFT analytic gradient. The hypercomplex method propagates through the fixed-point iteration in hypercomplex arithmetic, converging to the fixed point of the augmented system. The result is equivalent to applying IFT at convergence.

Table 4: HC gradient vs. IFT analytic gradient. Max $\|g_{\mathrm{HC}} - g_{\mathrm{IFT}}\|_\infty$ over all parameter entries.

n	Max gradient error	Status
3	2.80×10^{-14}	PASS
5	1.87×10^{-13}	PASS
8	7.17×10^{-13}	PASS

All errors are below 10^{-12}, consistent with machine-precision arithmetic. The fixed-point residual before evaluation is below 10^{-15} in all cases.

Claim 2: HC runtime is competitive. HC computes the full Hessian in one forward pass through the fixed-point iteration. JAX unrolled AD with $T = 500$ iterations differentiates a 500-step computational graph.

Table 5: HC runtime vs. JAX unrolled (500 steps). CPU, 5 repetitions, median.

n	HC (ms)	JAX unrolled (ms)	Speedup
3	11.5	300–1200	27–103×
5	49.0	1000–4000	20–81×
8	292.2	5000–8000	17–27×

Claim 3: JAX unrolled $\neq$ IFT Hessian. When both HC and JAX unrolled are available, we measure $\|H_{\mathrm{HC}} - H_{\mathrm{JAX}}\|_\infty$. For all tested sizes, this discrepancy is $O(1)$ or larger — systematically different results, not numerical noise. HC gives the IFT-correct Hessian; JAX gives the unrolled-iteration Hessian.

The practical implication: any second-order optimizer applied to implicit layers using JAX unrolled AD is operating on an incorrect Hessian. The hypercomplex method is the only single-pass approach that recovers the theoretically correct quantity.

5.3 Trust-Region Optimization

Table 6 demonstrates the convergence benefit of exact Hessians on a modified Rosenbrock function $f(x, y) = (1 - x)^2 + 10(y - x^2)^2$ with an added rotation.

The diagonal approximation (as used by Adam, RMSProp, and similar methods) discards off-diagonal curvature. On a rotated basin, the off-diagonal Hessian is ~ 40 at the starting point — the dominant term — which diagonal methods never capture. Exact and finite-difference Hessians converge in 16 iterations; the diagonal approximation stalls at $f = 6.2 \times 10^{-3}$ after 150 iterations and never reaches $f < 10^{-6}$.

Table 6: Trust-region convergence: exact Hessian (hcderiv) vs. finite differences vs. diagonal approximation.

Method	Iterations	Accepted	Rejected	Final f
Exact (hcderiv)	16	14	2	1.8×10^{-27}
Finite differences	16	14	2	4.0×10^{-18}
Diagonal approx.	150	139	11	6.2×10^{-3}

6 Library Architecture

6.1 Public API

Table 7: hcderiv public API.

Function	Description
`grad(f, x)`	Exact gradient vector
`hessian(f, x)`	Exact $d \times d$ Hessian matrix
`grad_and_hessian(f, x)`	Both simultaneously
`jacobian(f, x)`	Jacobian of vector-valued f
`hessian_vector_product(f, x, v)`	$H(\mathbf{x})\mathbf{v}$
`ridge_curvature(f, x)`	$\lambda_{\max}(H)$
`principal_curvatures(f, x)`	Eigenvalues and eigenvectors of H
`curvature_map(f, xs, ys)`	Ridge curvature over 2D grid
`shape_operator(f, x)`	Weingarten map of level set
`hessian_xla_jit`	`jax.jit`-compiled end-to-end Hessian

All functions accept `backend="numpy"` (default), `"jax"`, or `"jax-xla"`.

6.2 Backend Summary

Table 8: Backend selection guide.

Backend	Speed	JAX-composable	Use when
`numpy`	Fast (ms range)	No	Scientific computing, optimization
`jax`	Slower (Python dispatch)	Yes	JAX pipelines, composability
`jax-xla`	Fast (ms range)	Yes	JAX pipelines, JIT, XLA

6.3 Test Suite

hcderiv v0.4.0 ships with 117 tests:

Table 9: Test suite breakdown.

Test module	Count
NumPy core (vectorized algebra, unary ops, API)	80
JAX backend (eager dispatch, precision)	21
Layout and algebra correctness	16
JAX-XLA (tensor structure, Rosenbrock, JIT stability)	37
Total	**117** (all passing)

7 Runtime Trace: Coefficient Flow Through a Forward Pass

To make the mechanism concrete, Figure **??** sketches the coefficient flow through a single hypercomplex forward evaluation for $f(x) = \sin(x_1) \cdot \exp(x_2)$ at $d = 2$.

Each input is augmented as $X_j = x_j + \varepsilon_j$, where ε_j is an infinitesimal basis element satisfying $\varepsilon_j^2 = 0$ and $\varepsilon_j \varepsilon_k = \varepsilon_{jk}$ for $j \neq k$. The coefficient vector has size $1 + d + d(d+1)/2 = 1 + 2 + 3 = 6$:

$$X_1 = [x_1,\ 1,\ 0,\ 0,\ 0,\ 0] \tag{4}$$

$$X_2 = [x_2,\ 0,\ 1,\ 0,\ 0,\ 0] \tag{5}$$

After $\sin(X_1)$ and $\exp(X_2)$, each coefficient channel carries the corresponding partial derivative. Multiplication via the precomputed tensor M combines them. Extraction at the end:

- Primal: `coeffs[0]` $= f(\mathbf{x})$
- Gradient: `coeffs[1:3]` $= [\partial f/\partial x_1,\ \partial f/\partial x_2]$
- Hessian diagonal: `2 * coeffs[3:5]`
- Off-diagonal: `coeffs[5]` $= \partial^2 f/\partial x_1 \partial x_2$

The key property is that no separate backward pass is needed. The algebra propagates all derivative information forward simultaneously, at the cost of operating on coefficient vectors of size $O(d^2)$ rather than scalars. For $d \leq 64$, this is substantially faster than constructing a computation graph and running reverse-mode AD twice.

In the XLA backend, the entire coefficient vector is a single JAX array and the multiplication tensor M is a precomputed JAX constant. Every arithmetic operation in the function body becomes a `jnp.einsum` call. The full pipeline — inputs to extracted Hessian — is a single JIT-compiled XLA program.

8 Ecosystem Integration

hcderiv is the computational engine for the Mythos-Class Security Stack:

- **CARE** [3] uses hcderiv to compute exact Hessians of a risk potential $R(\mathbf{x})$ in real time, enabling curvature-based drift detection for deployed AI systems.
- **Constitutional OS** [4] consumes curvature signals as governance inputs via its canary-drift membrane (M5), using UAG Theorem 4 to identify soft escape directions.

- **UAG** [5] provides the theoretical basis for why ridge curvature is the relevant quantity for system stability and transition analysis.

The `ridge_curvature` and `principal_curvatures` functions are specifically designed for this use case, returning $\lambda_{\min}(H)$ (the softest direction) rather than $\lambda_{\max}$.

9 Related Work

Forward-mode AD. Forward-mode AD [2] computes directional derivatives in one pass but requires d passes for the full Hessian. Hypercomplex perturbation generalizes this by encoding $d(d+1)/2$ second-order terms in a single pass at the cost of a larger algebra.

BackPACK. BackPACK [7] extends PyTorch's `backward()` to compute approximate curvature information (diagonal, block-diagonal, Kron factorizations) during the backward pass. BackPACK is specialized for layer-wise curvature in neural networks; hcderiv is general-purpose and exact but does not integrate directly with PyTorch's autograd graph.

ViViT. ViViT [6] provides computationally tractable Gram matrices for second-order information in deep learning. hcderiv provides the complementary capability of exact full Hessians for small to medium dimensions, where the full matrix is tractable.

JAX `jax.hessian`. JAX's native `jax.hessian` uses forward-over-reverse AD and requires the function to be expressed in JAX primitives. hcderiv supports arbitrary Python callables that use the Hyper algebra, including fixed-point iterations that cannot be expressed as JAX primitives without unrolling.

Complex-step method. The complex-step method [8] uses imaginary perturbation to recover gradients without cancellation error. hcderiv extends this to second-order by using a richer algebra that encodes Hessian terms.

10 Conclusion

We have presented hcderiv v0.4.0, a library for exact one-pass gradient and Hessian computation via hypercomplex perturbation algebra. The JAX-XLA backend achieves 137–260$\times$ speedup over the v0.3 JAX backend while enabling end-to-end `jax.jit` compilation. For implicit layers, the hypercomplex method is the only single-pass approach that recovers the IFT-correct Hessian; JAX unrolled AD produces a systematically different result by differentiating through iteration history. hcderiv is available as a pip-installable library and serves as the curvature engine for a broader ecosystem including CARE, Constitutional OS, and the Mythos containment architecture.

Companion Artifacts

- **GitHub:** https://github.com/zetta55byte/hypercomplex
- **PyPI:** https://pypi.org/project/hcderiv/ (`pip install "hcderiv[jax]"`)
- **Software DOI:** 10.5281/zenodo.19389522
- **UAG paper:** 10.5281/zenodo.19448508
- **CARE v0.2.0:** https://github.com/zetta55byte/care
- **Mythos architecture:** 10.5281/zenodo.19464889

References

[1] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[2] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18:1–43, 2018.

[3] Zetta Byte. CARE v0.2.0: Curvature-aware risk engine. GitHub, 2026.

[4] Zetta Byte. Constitutional OS: A formal governance substrate for AI ecosystems, 2026.

[5] Zetta Byte. The Unified Attractor Grammar: A scale-free generative theory of stability, curvature, and emergence, 2026.

[6] Felix Dangel, Stefan Harmeling, and Philipp Hennig. ViViT: Curvature access through the generalized Gauss-Newton's low-rank structure. In *International Conference on Learning Representations*, 2022.

[7] Felix Dangel, Frederik Kunstner, and Philipp Hennig. BackPACK: Packing more into backprop. In *International Conference on Learning Representations*, 2020.

[8] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. The complex-step derivative approximation. *ACM Transactions on Mathematical Software*, 29(3):245–262, 2003.

[9] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.